

syNNapse PS-1

Vision-Based Inventory Re-Identification

ResNet-50 → Swin-T → DINOv2 Benchmark Study

Dataset	Stanford Online Products (SOP)
Framework	PyTorch, FAISS, FastAPI
Backbones	ResNet-50, Swin-Tiny, DINOv2 ViT-S/14
Loss	ArcFace CE + Batch-Hard Triplet
Deployment	FastAPI REST API + Hugging Face Hub

Contents

1	Problem Statement	3
1.1	Overview	3
1.2	Formal Problem Definition	3
1.3	Key Challenges	3
1.4	Evaluation Protocol	3
2	Dataset	4
2.1	Stanford Online Products (SOP)	4
3	System Architecture & Pipeline	4
3.1	High-Level Pipeline	4
3.2	Data Loading: SOPDataset and PKBatchSampler	5
3.2.1	SOPDataset	5
3.2.2	PKBatchSampler	5
3.3	Data Augmentation	5
3.3.1	Training Augmentation	5
3.3.2	Test-Time Augmentation (TTA)	5
4	Model Architectures	5
4.1	Shared Components	6
4.1.1	Generalised Mean (GeM) Pooling	6
4.1.2	ArcFace Classification Head	6
4.1.3	Projection Head	6
4.2	Model A: ResNet-50 (CNN Baseline)	6
4.3	Model B: Swin Transformer Tiny	7
4.4	Model C: DINOv2 ViT-S/14 (Primary Model)	7
5	Loss Function Design	7
5.1	Combined Metric Learning Loss	7
5.2	Batch-Hard Triplet Loss	7
5.3	ArcFace Cross-Entropy	8
6	Training Procedure	8
6.1	Optimiser and Learning Rate Schedule	8
6.2	Regularisation and Stability	8
6.3	Training Loop	8
7	Retrieval System	9
7.1	FAISS Gallery Index	9
7.2	Evaluation Protocol	9
8	Modular Codebase (src/ Package)	10
9	API Deployment	10
9.1	FastAPI REST Endpoint	10
9.2	Lazy Initialisation	10
9.3	Hugging Face Hub Integration	10

10 Design Decisions and Justifications	11
11 Environment and Dependencies	11

Problem Statement

Overview

Inventory management in large-scale e-commerce and warehouse environments faces a fundamental computer vision challenge: given an image of an item captured under arbitrary lighting, angle, or background conditions, the system must identify and retrieve the same or visually similar items from a potentially massive product catalogue. This task is known as **instance-level image retrieval** or **visual re-identification (Re-ID)**.

Formal Problem Definition

Let $\mathcal{G} = \{(x_i, y_i)\}_{i=1}^N$ be a **gallery database** of N product images, where each image x_i is associated with a unique item identifier y_i . Given a **query image** x_q (photographed in-the-wild), the task is to retrieve the top- K most similar items from $\mathcal{G}$:

$$\hat{Y}_q = \text{top-}K_{i \in \{1, \dots, N\}} \text{sim}(f(x_q), f(x_i))$$

where $f : \mathcal{X} \rightarrow \mathbb{R}^d$ is a learned embedding function that projects images into a compact metric space, and $\text{sim}(\cdot, \cdot)$ denotes cosine similarity.

Key Challenges

- **Open-set recognition:** At inference time, the model encounters items not seen during training. The system must generalise via a learned metric rather than class-specific memorisation.
- **Intra-class variation:** The same product can appear under varied illumination, scale, occlusion, and viewpoint.
- **Inter-class similarity:** Visually similar products (e.g., two chairs of the same design but different colours) must still be distinguished at the instance level.
- **Scalability:** Retrieval must remain efficient as the gallery grows to tens of thousands of items.
- **Domain specificity:** General-purpose features from ImageNet-pretrained models may not capture product-level discriminative cues.

Evaluation Protocol

Performance is measured using **Top- K Retrieval Accuracy** in an open-set setting. For each unique item in the test split, one image is selected as the query; the remainder form the gallery. Self-matches are excluded. A retrieval is considered correct if any image of the same item appears within the top- K ranked results. The primary metric is **Top-5 accuracy**, consistent with the problem-set specification.

Dataset

Stanford Online Products (SOP)

The experiments are conducted on the **Stanford Online Products** dataset, a widely-used benchmark for metric learning and image retrieval. The dataset contains product images scraped from eBay across 12 product super-categories.

Table 1: Stanford Online Products — Dataset Statistics

Split	Images	Item Classes	Avg. img/class
Train	~59,551	11,318	~5.3
Test	~60,502	11,316	~5.3
Total	~120,053	22,634	~5.3

Key properties:

- Train and test item IDs are *disjoint* (zero overlap), enforcing a true open-set evaluation.
- Images vary significantly in resolution, background clutter, and viewpoint.
- 12 super-categories include bicycles, cabinets, chairs, kettles, lamps, mugs, sofas, tables, fans, irons, and more.

The dataset is accessed programmatically via `kagglehub` and parsed from the official `Ebay_train.txt` / `Ebay_test.txt` manifest files.

System Architecture & Pipeline

High-Level Pipeline

The end-to-end system follows a metric learning paradigm with a gallery-based retrieval stage at inference time:

SOP Dataset $\rightarrow$ PKBatchSampler $\rightarrow$ Backbone
 $\rightarrow$ 512-d Projection + L2 Normalisation $\rightarrow$ ArcFace Head
 $\rightarrow$ Triplet + CE Loss $\rightarrow$ FAISS Gallery Index $\rightarrow$ Top-K Retrieval

The pipeline is decomposed into the following stages:

1. **Data preparation:** manifest parsing, label encoding, sanity checks.
2. **Data loading:** augmentation-aware `SOPDataset` + structured `PKBatchSampler`.
3. **Feature extraction:** backbone forward pass + GeM pooling + projection head.
4. **Metric learning:** combined ArcFace + Batch-Hard Triplet loss optimisation.
5. **Retrieval:** FAISS inner-product nearest-neighbour search over gallery embeddings.
6. **Evaluation:** open-set Top-K accuracy with self-exclusion.
7. **Deployment:** FastAPI REST endpoint with Hugging Face Hub model hosting.

Data Loading: SOPDataset and PKBatchSampler

3.2.1 SOPDataset

A custom `torch.utils.data.Dataset` subclass that serves:

- **Training mode:** returns (`image_tensor`, `label_int`) pairs.
- **Evaluation mode:** returns (`image_tensor`, `item_id_int`, `image_path`) triplets (path retained for gallery construction).

3.2.2 PKBatchSampler

A structured batch sampler that yields batches containing exactly P classes with K samples each, giving a total batch size of $P \times K$. This structure is critical for the triplet loss formulation: it guarantees that hard positive and hard negative pairs exist within every batch.

Configuration used: $P = 24$ classes, $K = 4$ images per class $\Rightarrow$ batch size of 96.

The sampler filters out classes with fewer than K available images and shuffles class order at each epoch to prevent ordering bias.

Data Augmentation

3.3.1 Training Augmentation

A strong augmentation policy is applied to improve generalisation:

- **RandomResizedCrop** (scale 0.6–1.0, ratio 0.75–1.33) $\rightarrow$ forces scale and viewpoint invariance.
- **RandomHorizontalFlip** ($p = 0.5$).
- **ColorJitter** (brightness, contrast, saturation ± 0.3 ; hue ± 0.05) $\rightarrow$ illumination robustness.
- **RandomGrayscale** ($p = 0.05$) $\rightarrow$ forces shape-based feature learning.
- **RandomErasing** ($p = 0.4$, scale 0.02–0.2) $\rightarrow$ occlusion robustness.
- **ImageNet normalisation** ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$).

3.3.2 Test-Time Augmentation (TTA)

At evaluation, two augmented views (centre-crop and horizontally-flipped centre-crop) are extracted per image. The model encodes both views and the resulting embedding vectors are averaged before L2 normalisation. This yields a 1–2% Top-5 improvement at zero additional training cost.

Model Architectures

Three backbone architectures are benchmarked, all sharing the same downstream projection head and classification head components.

Shared Components

4.1.1 Generalised Mean (GeM) Pooling

Standard average pooling aggregates spatial feature maps uniformly. **GeM pooling** introduces a learnable exponent p that interpolates between average pooling ($p = 1$) and max pooling ($p \rightarrow \infty$):

$$\text{GeM}(x) = \left(\frac{1}{|S|} \sum_{i \in S} x_i^p \right)^{1/p}$$

The parameter p is initialised to 3.0 and learned during training. GeM emphasises distinctive high-activation regions over background, improving retrieval performance compared to global average pooling. This is applied to the ResNet-50 backbone in place of its standard `avgpool` layer.

4.1.2 ArcFace Classification Head

To maximise discriminative power in the embedding space, the standard linear classification head is replaced by an **ArcFace** margin head. ArcFace adds an angular margin m to the target class angle before computing the softmax:

$$\mathcal{L}_{\text{ArcFace}} = -\log \frac{e^{s \cdot \cos(\theta_{y_i} + m)}}{e^{s \cdot \cos(\theta_{y_i} + m)} + \sum_{j \neq y_i} e^{s \cdot \cos \theta_j}}$$

where s is a feature scale factor and θ_{y_i} is the angle between the embedding and the target class weight vector. This formulation enforces a strict angular margin between classes, producing more separable and generalisable embeddings.

Configuration: $s = 64.0$, $m = 0.35$, Xavier-initialised weight matrix.

4.1.3 Projection Head

A lightweight projection head maps backbone features to a 512-dimensional embedding space:

$$\text{Backbone features} \xrightarrow{\text{LayerNorm or BN}} \text{Linear}(d_{\text{in}}, 512) \xrightarrow{\text{BN}} \text{L2-normalise} \in \mathbb{R}^{512}$$

Model A: ResNet-50 (CNN Baseline)

- **Backbone:** ResNet-50 pretrained on ImageNet-1K (V2 weights).
- **Pooling:** Default `avgpool` replaced with `GeMPooling(p=3.0)`.
- **Feature dim:** 2048 $\rightarrow$ projected to 512.
- **Training:** Layer4 of ResNet fine-tuned at $\text{lr}_{\text{bb}} = 3 \times 10^{-5}$; projection and ArcFace head at $\text{lr}_{\text{head}} = 3 \times 10^{-4}$.
- **Role:** Establishes a strong CNN baseline with standard convolutional inductive biases.

Model B: Swin Transformer Tiny

- **Backbone:** `swin_tiny_patch4_window7_224` from `timm`, pretrained on ImageNet-1K.
- **Pooling:** Swin Transformer returns a globally pooled token natively (no explicit GeM needed).
- **Feature dim:** $768 \rightarrow$ projected to 512.
- **Training:** Full backbone fine-tuned at $\text{lr}_{\text{bb}} = 2 \times 10^{-5}$; head at 3×10^{-4} .
- **Role:** Introduces hierarchical windowed self-attention, bridging CNNs and full ViT models.

Model C: DINOv2 ViT-S/14 (Primary Model)

- **Backbone:** `dinov2_vits14` loaded from Facebook Research via `torch.hub`.
- **Self-supervised pretraining:** DINOv2 is pretrained with a distillation-based self-supervised objective on a large curated dataset (LVD-142M), producing semantically rich and spatially aware CLS token features without relying on human labels.
- **Fine-tuning strategy:** All backbone parameters are initially frozen. Only the **last 4 transformer blocks** and the final **LayerNorm** are unfrozen for domain adaptation. This prevents catastrophic forgetting while allowing task-specific feature refinement.
- **Feature dim:** 384 (CLS token) $\rightarrow$ projected to 512.
- **Training:** Unfrozen backbone layers at $\text{lr}_{\text{bb}} = 1 \times 10^{-5}$; projection and ArcFace head at 3×10^{-4} .
- **Role:** Primary model; exploits robust self-supervised features for open-set generalisation.

Loss Function Design

Combined Metric Learning Loss

The training objective combines two complementary losses:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{triplet}} \cdot \mathcal{L}_{\text{triplet}} + \lambda_{\text{CE}} \cdot \mathcal{L}_{\text{ArcFace}}$$

with $\lambda_{\text{triplet}} = 1.0$ and $\lambda_{\text{CE}} = 0.3$.

Batch-Hard Triplet Loss

The triplet loss enforces that an anchor embedding a is closer to a positive p (same class) than to a negative n (different class) by a margin α :

$$\mathcal{L}_{\text{triplet}} = \max(0, \|f(a) - f(p)\|_2 - \|f(a) - f(n)\|_2 + \alpha)$$

The **batch-hard mining** strategy (from `pytorch-metric-learning`) selects the hardest positive (furthest same-class sample) and hardest negative (closest different-class sample)

within each batch. This focuses gradient updates on the most informative pairs, greatly accelerating metric space learning. Margin: $\alpha = 0.3$.

ArcFace Cross-Entropy

The ArcFace CE loss (with **label smoothing** $\epsilon = 0.1$) provides stable class-level supervision and prevents the model from becoming overconfident on training identities. It acts as a regulariser for the triplet loss, anchoring the metric space to explicit class prototypes while the triplet loss shapes fine-grained intra-class compactness.

Rationale: Triplet loss alone can be slow to converge and is sensitive to batch composition. ArcFace CE provides a global class structure. The combined objective leverages the complementary strengths of both.

Training Procedure

Optimiser and Learning Rate Schedule

All models are optimised with **AdamW** using differential learning rates (lower for the backbone, higher for the projection and ArcFace heads) and weight decay $\lambda = 10^{-4}$.

The learning rate follows a **OneCycleLR** schedule:

- Warm-up phase: 10% of total steps, linearly increasing from 0 to $\text{lr}_{\max}$.
- Annealing phase: cosine decay from $\text{lr}_{\max}$ to ≈ 0 .

This approach has been shown to converge faster than step-based or exponential schedules, reduces sensitivity to initial learning rate selection, and enables higher peak learning rates with stable training.

Regularisation and Stability

- **Gradient clipping:** Maximum norm of 5.0, preventing gradient explosion in deep transformer layers.
- **BatchNorm:** Applied in projection heads for training stability.
- **L2 normalisation:** All embedding vectors are L2-normalised before loss computation and retrieval, constraining the metric to the unit hypersphere and ensuring cosine similarity is equivalent to inner product.
- **Reproducibility:** Global seed (42) set across Python, NumPy, and PyTorch; cuDNN determinism enabled.

Training Loop

Each epoch proceeds as follows:

1. Sample batches via **PKBatchSampler** (PKsampling ensures every batch has P classes with K images each).
2. Forward pass: backbone $\rightarrow$ projection $\rightarrow$ L2-normalise $\rightarrow$ ArcFace head.

3. Compute combined loss (batch-hard triplet mining + ArcFace CE with label smoothing).
4. Backward pass, gradient clipping, optimiser step, scheduler step.
5. Save epoch checkpoint atomically (write to temp, then move) to Google Drive.
6. Every `eval_every` epochs: extract all test embeddings, run FAISS retrieval, report Top-1/5/10.
7. Track best Top-5 accuracy; save “best” checkpoint; increment early-stopping counter.
8. Early stopping triggers after `patience` evaluation cycles without improvement.

Training configurations:

Model	Epochs	Patience	Eval every
ResNet-50	10	4	2
Swin-Tiny	10	4	2
DINOv2 ViT-S	12	5	2

Retrieval System

FAISS Gallery Index

After training, embeddings for all test images are extracted using the fine-tuned DINOv2 model (with TTA). These are stored as a **FAISS IndexFlatIP** (inner product index), which performs exact nearest-neighbour search in $O(N \cdot d)$ time.

Since all embeddings are L2-normalised, inner-product search is mathematically equivalent to cosine similarity:

$$\text{sim}(q, g) = \langle \hat{f}(q), \hat{f}(g) \rangle = \cos \theta_{qg}$$

Gallery assets (embeddings `.npy`, item IDs, and image paths) are persisted to disk and can be reloaded without re-running inference.

Evaluation Protocol

The `topk_accuracy` function implements the PS-aligned open-set evaluation:

1. For each unique item ID in the test set with ≥ 2 images, randomly select one as the query.
2. Add the full embedding matrix to the FAISS index.
3. Retrieve top- $(K + 50)$ neighbours (extra buffer to handle self-exclusion).
4. Remove the query’s own index from the retrieved list (self-exclusion).
5. A hit at rank k : the ground-truth item ID appears in the top- k retrieved IDs.
6. Report accuracy as fraction of queries with at least one correct match in top- K .

Modular Codebase (src/ Package)

The notebook generates a production-ready Python package alongside the training code:

- `src/build_model.py` — Factory function `build_model(variant, num_classes, emb_dim, ckpt_path, device)` that instantiates any of the three model variants by name string and optionally loads a checkpoint. Enables clean deployment without coupling to notebook code.
- `src/feature_extraction/encoder.py` — `ImageEncoder` class: high-level wrapper accepting a PIL image and returning a 512-d L2-normalised NumPy vector. Encapsulates all preprocessing, device management, and inference logic.
- `src/feature_extraction/build_gallery.py` — `build_gallery_database()`: iterates a `DataLoader`, encodes all images, and persists `.npz` gallery files. Designed to be run once after training.
- `src/similarity_scoring_and_retrieval/retriever.py` — `FaissRetriever` class: loads pre-built gallery `.npz` files, builds a FAISS index at construction time, and exposes a `search(q_emb, k)` method returning top- k item IDs, similarity scores, indices, and reference paths.

API Deployment

FastAPI REST Endpoint

A **FastAPI** application (`src/api/main.py`) wraps the inference pipeline and exposes two endpoints:

- `GET /health` — Returns model readiness status, device type (CUDA/CPU), gallery size, and server uptime. Used for liveness and readiness probes.
- `POST /search` — Accepts a multipart image upload and a query parameter k (default 5, range 1–50). Returns a JSON array of ranked results, each containing `item_id`, cosine `score`, and reference path.

Lazy Initialisation

Model loading and gallery construction are deferred to the first request (protected by a threading lock). This avoids long startup times for cold-start deployments while ensuring thread-safe singleton initialisation.

Hugging Face Hub Integration

The deployment script `scripts/upload_assets_to_hf.py` uploads:

- `dinov2_vits14_best.pth` — best model checkpoint.
- `gallery_embeddings.npy`, `gallery_item_ids.npy`, `gallery_refs.npy` — pre-built gallery index files.

The API server automatically downloads missing assets from the Hugging Face repository on first startup, enabling zero-friction deployment on platforms such as Hugging Face Spaces, Google Cloud Run, or any containerised environment.

Design Decisions and Justifications

1. **Open-set evaluation over closed-set:** The test item IDs are disjoint from training IDs, reflecting the real-world scenario where new products are continuously added. Closed-set accuracy would be misleadingly optimistic.
2. **DINOv2 as primary model:** DINOv2 is pretrained on a 142M-image curated dataset using a self-supervised distillation objective, yielding features with strong semantic and geometric understanding. This provides a significantly richer initialisation for open-set retrieval than supervised ImageNet pretraining.
3. **Selective unfreezing:** Fully fine-tuning DINOv2 risks catastrophic forgetting of rich pre-trained representations. Unfreezing only the last 4 transformer blocks balances domain adaptation with feature preservation.
4. **ArcFace over plain softmax:** ArcFace enforces geometric inter-class separability in angular space rather than only optimising for classification accuracy, directly improving retrieval generalisation.
5. **GeM over AvgPool:** GeM’s learned aggregation exponent emphasises discriminative spatial regions, which is especially beneficial for retrieval tasks where salient texture and shape features must be captured.
6. **FAISS for retrieval:** CPU FAISS with `IndexFlatIP` provides exact nearest-neighbour retrieval with sub-second latency for gallery sizes in the tens of thousands, without requiring a vector database server.
7. **Atomic checkpointing:** Write to a temp file, then `shutil.move` ensures no corrupted checkpoint is ever persisted, even if the Colab session terminates mid-write.

Environment and Dependencies

Table 2: Core Dependencies

Library	Role
<code>torch ≥ 2.1.0</code>	Deep learning framework
<code>torchvision ≥ 0.16.0</code>	Pretrained models, transforms
<code>timm ≥ 0.9.0</code>	Swin Transformer backbone
<code>faiss-cpu ≥ 1.7.4</code>	Approximate nearest-neighbour search
<code>pytorch-metric-learning ≥ 2.3.0</code>	Batch-hard miner, triplet loss
<code>kagglehub</code>	Dataset download
<code>fastapi ≥ 0.110.0</code>	REST API framework
<code>uvicorn</code>	ASGI server
<code>huggingface_hub</code>	Model and asset hosting
<code>numpy, pandas, matplotlib, tqdm, Pillow</code>	Scientific computing utilities

The project runs on Google Colab with a GPU runtime (T4/A100). The `torch.hub` mechanism is used to download DINOv2 weights directly from `facebookresearch/dinov2`.

